

Testing Strategy & Coverage Plan — Family Music Scrobbler PWA

Testing Strategy & Coverage Plan - Family Music Scrobbler PWA

A complete testing strategy for ensuring reliability, correctness, and long-term maintainability of the Family Music Scrobbler PWA. This document defines testing layers, tools, coverage targets, CI integration, and guidelines for both backend and frontend testing.

1. Purpose of This Document

This strategy ensures:

- Confidence in every change
- Predictable releases
- Protection against regressions
- Full coverage of ingestion, API, PWA offline behaviour, and UI flows

It is designed to work seamlessly with your CI/CD pipeline.

2. Testing Layers

The project uses a **multi-layer testing model**:

1. Unit Tests

Small, isolated tests for functions, components, and logic.

2. Integration Tests

Tests that validate interactions between modules.

3. End-to-End (E2E) Tests

Full system tests simulating real user behaviour.

4. PWA Offline Tests

Tests for service worker, caching, and background sync.

5. Load & Performance Tests

Ensures ingestion and API endpoints scale.

3. Backend Testing Strategy (FastAPI)

3.1 Unit Tests

Test:

- Models
- Schemas
- Utility functions
- Token encryption/decryption
- Backoff logic

Tools:

- pytest
- pytest-asyncio
- mypy

3.2 Integration Tests

Test:

- API endpoints
- Database interactions
- Ingestion worker logic

Use a **temporary PostgreSQL container** for integration tests.

3.3 Ingestion Worker Tests

Test:

- Spotify API mock responses
- Deduplication logic
- Partition routing
- Token refresh lifecycle

- Backoff behaviour

Mock Spotify API using:

- responses
- pytest-mock

3.4 Coverage Targets

- Models: 100%
 - API endpoints: 90%+
 - Ingestion worker: 85%+
 - Utility functions: 100%
-

4. Frontend Testing Strategy (React PWA)

4.1 Unit Tests

Test:

- Components
- Hooks
- Utility functions

Tools:

- vitest
- @testing-library/react

4.2 Integration Tests

Test:

- Page-level flows
- API wrapper (lib/api.js)
- IndexedDB operations

4.3 PWA Offline Tests

Test:

- Service worker registration
- Cache-first behaviour
- Network-first fallback
- Background sync queue replay

Use:

- workbox test utilities
- Browser devtools automation

4.4 E2E Tests

Simulate real user behaviour:

- Login
- View recent tracks
- Edit profile
- Offline mode
- Reconnection sync

Tools:

- Playwright
- Cypress

4.5 Coverage Targets

- Components: 90%+
- Hooks: 90%+
- Service worker: 80%+
- IndexedDB logic: 85%+

5. API Contract Testing

Ensure frontend and backend stay in sync.

Test:

- Response shapes
- Error formats
- Pagination
- Authentication flows

Tools:

- schemathesis
- OpenAPI validation

6. Database Testing Strategy

6.1 Partition Tests

Test:

- Correct partition creation
- Inserts routed to correct partition
- Queries across partitions

6.2 Constraint Tests

Test:

- UNIQUE(user_id, spotify_play_id)
- Foreign keys
- NOT NULL constraints

6.3 Migration Tests

Test:

- Alembic migrations apply cleanly
- Rollbacks work



7. CI/CD Integration

CI Runs:

- Backend unit tests
- Frontend unit tests
- Integration tests
- Linting
- Type checking

CD Runs:

- Build Docker images
- Run smoke tests
- Deploy to NUC

Smoke Tests Include:

- /healthz endpoint
- Database connectivity
- Service worker registration

8. Test Data Strategy

Mock Data

Used for unit tests.

Seed Data

Used for integration tests.

Synthetic Scrobbles

Used for ingestion worker tests.

Realistic Spotify Payloads

Captured once, stored as fixtures.

9. Developer Workflow

Before committing:

- Run pytest
- Run npm test
- Run flake8
- Run eslint

Before merging:

- Ensure CI passes
- Review coverage report

Before deploying:

- Run smoke tests
-

10. Summary

This testing strategy ensures:

- High confidence in every change
- Protection against regressions
- Reliable ingestion
- Predictable PWA offline behaviour

- Strong API contract stability

It forms the backbone of long-term maintainability for the Family Music Scrobbler PWA.